← back to the lesson

Shuffle an array

importance: 3

Write the function `shuffle(array)` that shuffles (randomly reorders) elements of the array.

Multiple runs of `shuffle` may lead to different orders of elements. For instance:

```
let arr = [1, 2, 3];

shuffle(arr);
// arr = [3, 2, 1]

shuffle(arr);
// arr = [2, 1, 3]

shuffle(arr);
// arr = [3, 1, 2]
// ...
```

All element orders should have an equal probability. For instance, `[1,2,3]` can be reordered as `[1,2,3]` or `[1,3,2]` or `[3,1,2]` etc, with equal probability of each case.

solution

The simple solution could be:

```
function shuffle(array) {
  array.sort(() => Math.random() - 0.5);
}

let arr = [1, 2, 3];
shuffle(arr);
alert(arr);
```

That somewhat works, because `Math.random() - 0.5` is a random number that may be positive or negative, so the sorting function reorders elements randomly.

But because the sorting function is not meant to be used this way, not all permutations have the same probability.

For instance, consider the code below. It runs `shuffle` 1000000 times and counts appearances of all possible results:

```
function shuffle(array) {
  array.sort(() => Math.random() - 0.5);
}

// counts of appearances for all possible permutations
let count = {
  '123': 0,
  '132': 0,
  '213': 0,
  '231': 0,
  '321': 0,
  '312': 0
};

for (let i = 0; i < 1000000; i++) {
  let array = [1, 2, 3];
  shuffle(array);
  count[array.join('')]++;
}

// show counts of all possible permutations
for (let key in count) {
  alert(`${key}: ${count[key]}`);
}
```

An example result (depends on JS engine):

```
123: 250706
132: 124425
213: 249618
231: 124880
312: 125148
321: 125223
```

We can see the bias clearly: `123` and `213` appear much more often than others.

The result of the code may vary between JavaScript engines, but we can already see that the approach is unreliable.

Why it doesn't work? Generally speaking, `sort` is a "black box": we throw an array and a comparison function into it and expect the array to be sorted. But due to the utter randomness of the

comparison the black box goes mad, and how exactly it goes mad depends on the concrete implementation that differs between engines.

There are other good ways to do the task. For instance, there's a great algorithm called Fisher-Yates shuffle. The idea is to walk the array in the reverse order and swap each element with a random one before it:

```
function shuffle(array) {
  for (let i = array.length - 1; i > 0; i--) {
    let j = Math.floor(Math.random() * (i + 1)); // random index

    // swap elements array[i] and array[j]
    // we use "destructuring assignment" syntax to achieve that
    // you'll find more details about that syntax in later chapt
    // same can be written as:
    // let t = array[i]; array[i] = array[j]; array[j] = t
    [array[i], array[j]] = [array[j], array[i]];
  }
}
```

Let's test it the same way:

```
function shuffle(array) {
  for (let i = array.length - 1; i > 0; i--) {
    let j = Math.floor(Math.random() * (i + 1));
    [array[i], array[j]] = [array[j], array[i]];
  }
}

// counts of appearances for all possible permutations
let count = {
  '123': 0,
  '132': 0,
  '213': 0,
  '231': 0,
  '321': 0,
  '312': 0
};

for (let i = 0; i < 1000000; i++) {
  let array = [1, 2, 3];
  shuffle(array);
  count[array.join('')]++;
}

// show counts of all possible permutations
for (let key in count) {
  alert(`${key}: ${count[key]}`);
}
```

The example output:

```
123: 166693
132: 166647
213: 166628
231: 167517
312: 166199
321: 166316
```

Looks good now: all permutations appear with the same probability.

Also, performance-wise the Fisher-Yates algorithm is much better, there's no "sorting" overhead.

© 2007—2022 Ilya Kantorabout the projectcontact usterms of usageprivacy policy